

CSED211: Shell lab Report

컴퓨터공학과 20240940 장해영

1. Overview

해당 과제는 쉘의 작동방식을 이해하고 tiny shell을 구성하는 함수를 구현하고, signal에 대한 handler를 구현하는 과제이다.

2. eval

```
void eval(char *cmdline)
{
    char *argv[MAXARGS];
    char buf[MAXLINE];
    int bg;
    pid_t pid;
    sigset_t mask, prev;
    sigemptyset(&mask);
    sigaddset(&mask, SIGCHLD);

    strcpy(buf, cmdline);
    bg = parseline(buf, argv);

    // ignore empty lines
    if(argv[0] == NULL) return;
```

이 함수는 사용자에게 명령을 받아 처리하는 메인함수이다. Lecture 14 signals 8페이지의 코드를 참고하여 작성하였다. 이 함수에서 고려해야 할 사항은 다음과 같다. 우선, command line으로부터 받은 정보를 명령어와 인자로 파싱해야한다. 이는 helper function인 parseline을 이용하여 정보를 argv에 저장해주었다. 다음으로, 빈 정보가 내장명령어인 경우 builtin_cmd함수에서 이를 처리하고, 그렇지 않은 경우 시행하는 코드를 작성해주어야 한다.

```
// if not builtin cmd
if(!builtin_cmd(argv)){
    sigprocmask(SIG_BLOCK, &mask, &prev); // block SIGCHLD

    if((pid = fork()) == 0){ // child
        setpgid(0,0);
        sigprocmask(SIG_SETMASK, &prev, NULL); // unblock
        if(execve(argv[0], argv, environ) < 0){
            printf("%s: Command not found\n", argv[0]);
            exit(0);
        }
    }
}
```

이 때 writeup문서의 hint를 보면, 자식을 fork하기 전에 sigprocmask로 SIGCHLD를 block해야 한다고 나와있다. 이후 signal을 다시 unblock하고 sigprocmask로 addjob을 불러 joblist에 자식을 추가해준다. 자식

을 fork하는 동안 SIGCHLD가 전달되어 핸들러가 작동하는 것을 방지하는 것이다. 이 내용을 강의자료의 코드에 추가해주어야 한다. 이는 Lecture 14 signals 49페이지를 참고하여 작성하였다. 이 때 writeup문서를 참고하여 추가로 자식과 부모를 분리하기 위해 setpgid(0,0)을 추가해주었다.

```
// parent
else{
    if(bg){
        addjob(jobs, pid, BG, cmdline);
        sigprocmask(SIG_SETMASK, &prev, NULL); // unblock
        printf("[%d] (%d) %s", pid2jid(pid), pid, cmdline);
    } else {
        addjob(jobs, pid, FG, cmdline);
        sigprocmask(SIG_SETMASK, &prev, NULL); // unblock
        waitfg(pid);
    }
}
return;
```

이렇게 블록해제에 대한 코드를 추가해준 뒤 부모에 대한 코드를 추가해주었다. 부모는 background와 foreground를 구분하여 job을 추가해준다. Background의 경우 정보를 형식에 맞게 출력하고, foreground

의 경우 waitfg를 수행해 wait한다.

3. builtin_cmd

```
int builtin_cmd(char **argv)
{
    if(!strcmp(argv[0], "quit")){
        exit(0);
    }
    if(!strcmp(argv[0], "jobs")){
        listjobs(jobs);
        return 1;
    }
    if(!strcmp(argv[0], "bg") || !strcmp(argv[0], "fg")){
        do_bgfg(argv);
        return 1;
    }
    return 0; /* not a builtin command */
}
```

내장명령어를 판단하고, 이에 대한 수행을 하는 함수이다. writeup문서를 참고하면 내장 명령어에는 네가지가 있다. 쉘을 종료하는 quit, 작업을 background 또는 foreground에서 실행하는 bg, fg, background의 작업 목록을 출력하는 jobs이다.

우선 내장명령어를 판단하는 코드부터 짜도록 하자. writeup문서에 따르면 파싱한 문자열이 저장되어 있는 argv의 첫번째 값을 보면 이를 판단할 수 있다. 따라서 argv[0]의 값을 기준으로 수행해야 할 코드를 나누면 된다. strcmp함수를 이용하여 argv[0]값과 내장 명령어의 일치여부를 확인해주고, 이에 해당하지 않는다면, 내장명령어가 아니므로 return 0을 해주어 eval함수에서 fork를 진행할 수 있도록 해준다.

이제 각 내장 명령어의 기능에 대해 알아보자. 우선 quit의 경우 쉘을 종료하면 되므로 exit(0)을 해준다. 다음으로 jobs의 경우 모든 job의 리스트를 출력하면 되므로 helper function인 listjobs를 이용하여 출력해준다. 마지막으로 bg와 fg의 경우 추후 구현할 do_bgfg에서 처리해준다. 내장명령어인 경우 eval함수에서 fork를 진행하면 안되므로 return 1을 해준다.

4. do_bgfg

bg와 fg명령어를 처리하는 함수이다. 각각 background와 foreground에서 작업을

```
void do_bgfg(char **argv)
{
    struct job_t* job;
    int jid, pid;

    // no argument
    if(argv[1] == NULL){
        printf("%s command requires PID or %%jobid argument\n", argv[0]);
        return;
    }
}
```

실행하도록 해야한다. 먼저 전처리 과정을 거쳐야 한다. 옵션 뒤 argument가 없는 경우, tshref.out 파일을 참고하여 같은 형식의 안내 메시지를 출력하였다.

다음으로는 %로 시작하는 인자가 들어와 job id로 전달받는 경우를 고려해야 한

```
// job ID
if(argv[1][0] == '%'){
    jid = atoi(&argv[1][1]);
    job = getjobpid(jobs, jid);
    if(job == NULL){
        printf("%s: No such job\n", argv[1]);
        return;
    }
} else if(isdigit(argv[1][0])){ // process id
    pid = atoi(argv[1]);
    job = getjobpid(jobs, pid);
    if(job == NULL){
        printf("%s: No such process\n", argv[1]);
        return;
    }
} else { // error
    printf("%s: argument must be a PID or %%jobid\n", argv[0]);
    return;
}
```

다. 이 경우 getjobpid함수를 이용하여 입력받은 jid에 해당하는 job을 가져온다.

숫자로 시작하는 인자는 process id로 전달받는 경우 이므로, getjobpid함수를 이용하여 위와 같은 과정을 거친다. 만약 둘 다 해당되지 않는다면 tshref.out에 해당되는 에러메시지를 출력한다.

이제 옵션에 따라 기능을 구현할 차례이다. 우선 bg인 경우, background에서 다시 실행시켜야 하므로 SIGCONT를 보내 job을 다시 실행한 뒤 정보를 출력한다. Fg인 경우, foreground에서 다시 실행시켜야 하므로 마찬가지로 SIGCONT를 보내 job을 깨워주고 해당 프로세스가 종료될 때까지 wait해준다.

```
// bg
if(!strcmp(argv[0], "bg")){
    job->state = BG;
    kill(-job->pid, SIGCONT);
    printf("[%d] (%d) %s", job->jid, job->pid, job->cmdline);
}
// fg
else if(!strcmp(argv[0], "fg")){
    job->state = FG;
    kill(-job->pid, SIGCONT);
    waitfg(job->pid);
}
return;
```

5. waitfg

```
void waitfg(pid_t pid)
{
    while(pid == fgpid(jobs)){
        sleep(1);
    }
    return;
}
```

foreground에서 실행중인 프로세스가 종료될 때 까지 기다리게 하는 함수이다. Foreground에서 실행중인 프로세스가 종료되면 Writeup 문서에 나와있는 대로 sleep함수를 사용하였다. Lecture 14 signals 52페이지에 나와있는 sleep함수 사용법을 참고하여 1초동안 sleep하도록 하였다.

6. sigchld_handler

```
void sigchld_handler(int sig)
{
    int status;
    pid_t pid;

    while((pid = waitpid(-1, &status, WUNTRACED | WNOHANG)) > 0){
        if(WIFEXITED(status)){
            deletejob(jobs, pid);
        } else if(WIFSIGNALED(status)){
            printf("Job [%d] (%d) terminated by signal %d\n", pid2jid(pid), pid, WTERMSIG(status));
            deletejob(jobs, pid);
        } else if(WIFSTOPPED(status)){
            printf("Job [%d] (%d) stopped by signal %d\n", pid2jid(pid), pid, WSTOPSIG(status));
            getjobpid(jobs, pid)->state = ST;
        }
    }
    return;
}
```

이 함수 부터는 handler를 구현한다. SIGCHLD는 자식 프로세스가 terminate 또는 stop할 때 호출되는 시그널이다. 모든 좀비 자식 프로세스를 제거시키고

jobs상황을 업데이트 해야한다.

우선 writeup문서에 나와있는대로 waitpid를 WUNTRACED와 WNOHANG과 함께 사용할 것이다. waitpid함수의 사용법은 lecture13 exceptions and processes 56페이지와 교과서 745페이지를 참고하여 작성하였다. 첫번째 인자는 임의의 child에 대해 실행할 수 있도록 -1로 정하였다. 자식의 종료와 관련된 세개의 매크로를 사용하였다. WIFEXITED는 자식이 정상적으로 종료된 경우, WIFSIGNALED는 시그널을 받아 종료된 경우, WIFSTOPPED stop된 경우를 의미한다. 따라서 이에 따라 적절한 수행을 해준다. 정상적으로 종료된 경우, delete를 해주고, 시그널을 받아 종료된 경우 해당 내용을 출력하고 delete해주고, stop된 경우 해당 내용을 출력하고 state를 stop으로 바꿔준다. 출력형식은 tshref.out을 참고하였다.

7. sigint_handler

```
void sigint_handler(int sig)
{
    pid_t pid = fgpid(jobs);
    if (pid > 0) kill(-pid, sig);
    return;
}
```

해당함수는 사용자가 ctrl + c를 입력한 경우 호출된다. 현재 foreground에서 실행중인 작업을 terminate하면 된다. 현재 실행중인 프로세스를 찾아 SIGINT 시그널을 전달한다.

8. sigtstp_handler

```
void sigtstp_handler(int sig)
{
    pid_t pid = fgpid(jobs);
    if (pid > 0) kill(-pid, sig);
    return;
}
```

해당함수는 사용자가 ctrl + z를 입력한 경우 호출된다. 현재 foreground에서 실행중인 작업을 stop하면 된다. 현재 실행중인 프로세스를 찾아 SIGTSTP 시그널을 전달한다. trace

9. Result

A. Trace1

```
[hyjang@programming2 shlab-handout]$ make test01
./sdriver.pl -t trace01.txt -s ./tsh -a "-p"
#
# trace01.txt - Properly terminate on EOF.
#
[hyjang@programming2 shlab-handout]$ make rtest01
./sdriver.pl -t trace01.txt -s ./tshref -a "-p"
#
# trace01.txt - Properly terminate on EOF.
#
```

B. Trace2

```
[hyjang@programming2 shlab-handout]$ make test02
./sdriver.pl -t trace02.txt -s ./tsh -a "-p"
#
# trace02.txt - Process builtin quit command.
#
[hyjang@programming2 shlab-handout]$ make rtest02
./sdriver.pl -t trace02.txt -s ./tshref -a "-p"
#
# trace02.txt - Process builtin quit command.
#
```

C. Trace3

```
[hyjang@programming2 shlab-handout]$ make test03
./sdriver.pl -t trace03.txt -s ./tsh -a "-p"
#
# trace03.txt - Run a foreground job.
#
tsh> quit
[hyjang@programming2 shlab-handout]$ make rtest03
./sdriver.pl -t trace03.txt -s ./tshref -a "-p"
#
# trace03.txt - Run a foreground job.
#
tsh> quit
```

D. Trace4

```
[hyjang@programming2 shlab-handout]$ make test04
./sdriver.pl -t trace04.txt -s ./tsh -a "-p"
#
# trace04.txt - Run a background job.
#
tsh> ./myspin 1 &
[1] (11762) ./myspin 1 &
[hyjang@programming2 shlab-handout]$ make rtest04
./sdriver.pl -t trace04.txt -s ./tshref -a "-p"
#
# trace04.txt - Run a background job.
#
tsh> ./myspin 1 &
[1] (11772) ./myspin 1 &
```

E. Trace5

```
[hyjang@programming2 shlab-handout]$ make test05
./sdriver.pl -t trace05.txt -s ./tsh -a "-p"
#
# trace05.txt - Process jobs builtin command.
#
tsh> ./myspin 2 &
[1] (11805) ./myspin 2 &
tsh> ./myspin 3 &
[2] (11807) ./myspin 3 &
tsh> jobs
[1] (11805) Running ./myspin 2 &
[2] (11807) Running ./myspin 3 &
[hyjang@programming2 shlab-handout]$ make rtest05
./sdriver.pl -t trace05.txt -s ./tshref -a "-p"
#
# trace05.txt - Process jobs builtin command.
#
tsh> ./myspin 2 &
[1] (11819) ./myspin 2 &
tsh> ./myspin 3 &
[2] (11821) ./myspin 3 &
tsh> jobs
[1] (11819) Running ./myspin 2 &
[2] (11821) Running ./myspin 3 &
```

F. Trace6

```
[hyjang@programming2 shlab-handout]$ make test06
./sdriver.pl -t trace06.txt -s ./tsh -a "-p"
#
# trace06.txt - Forward SIGINT to foreground job.
#
tsh> ./myspin 4
Job [1] (11865) terminated by signal 2
[hyjang@programming2 shlab-handout]$ make rtest06
./sdriver.pl -t trace06.txt -s ./tshref -a "-p"
#
# trace06.txt - Forward SIGINT to foreground job.
#
tsh> ./myspin 4
Job [1] (11875) terminated by signal 2
```

G. Trace7

```
[hyjang@programming2 shlab-handout]$ make test07
./sdriver.pl -t trace07.txt -s ./tsh -a "-p"
#
# trace07.txt - Forward SIGINT only to foreground job.
#
tsh> ./myspin 4 &
[1] (11913) ./myspin 4 &
tsh> ./myspin 5
Job [2] (11915) terminated by signal 2
tsh> jobs
[1] (11913) Running ./myspin 4 &
[hyjang@programming2 shlab-handout]$ make rtest07
./sdriver.pl -t trace07.txt -s ./tshref -a "-p"
#
# trace07.txt - Forward SIGINT only to foreground job.
#
tsh> ./myspin 4 &
[1] (11927) ./myspin 4 &
tsh> ./myspin 5
Job [2] (11929) terminated by signal 2
tsh> jobs
[1] (11927) Running ./myspin 4 &
```

H. Trace8

```
[hyjang@programming2 shlab-handout]$ make test08
./sdriver.pl -t trace08.txt -s ./tsh -a "-p"
#
# trace08.txt - Forward SIGTSTP only to foreground job.
#
tsh> ./myspin 4 &
[1] (11978) ./myspin 4 &
tsh> ./myspin 5
Job [2] (11980) stopped by signal 20
tsh> jobs
[1] (11978) Running ./myspin 4 &
[2] (11980) Stopped ./myspin 5
[hyjang@programming2 shlab-handout]$ make rtest08
./sdriver.pl -t trace08.txt -s ./tshref -a "-p"
#
# trace08.txt - Forward SIGTSTP only to foreground job.
#
tsh> ./myspin 4 &
[1] (11991) ./myspin 4 &
tsh> ./myspin 5
Job [2] (11993) stopped by signal 20
tsh> jobs
[1] (11991) Running ./myspin 4 &
[2] (11993) Stopped ./myspin 5
```

I. Trace9

```
[hyjang@programming2 shlab-handout]$ make test09
./sdriver.pl -t trace09.txt -s ./tsh -a "-p"
#
# trace09.txt - Process bg builtin command
#
tsh> ./myspin 4 &
[1] (12063) ./myspin 4 &
tsh> ./myspin 5
Job [2] (12065) stopped by signal 20
tsh> jobs
[1] (12063) Running ./myspin 4 &
[2] (12065) Stopped ./myspin 5
tsh> bg %2
[2] (12065) ./myspin 5
tsh> jobs
[1] (12063) Running ./myspin 4 &
[2] (12065) Running ./myspin 5
[hyjang@programming2 shlab-handout]$ make rtest09
./sdriver.pl -t trace09.txt -s ./tshref -a "-p"
#
# trace09.txt - Process bg builtin command
#
tsh> ./myspin 4 &
[1] (12097) ./myspin 4 &
tsh> ./myspin 5
Job [2] (12099) stopped by signal 20
tsh> jobs
[1] (12097) Running ./myspin 4 &
[2] (12099) Stopped ./myspin 5
tsh> bg %2
[2] (12099) ./myspin 5
tsh> jobs
[1] (12097) Running ./myspin 4 &
[2] (12099) Running ./myspin 5
```

J. Trace10

```
[hyjang@programming2 shlab-handout]$ make test10
./sdriver.pl -t trace10.txt -s ./tsh -a "-p"
#
# trace10.txt - Process fg builtin command.
#
tsh> ./myspin 4 &
[1] (12192) ./myspin 4 &
tsh> fg %1
Job [1] (12192) stopped by signal 20
tsh> jobs
[1] (12192) Stopped ./myspin 4 &
tsh> fg %1
tsh> jobs
[hyjang@programming2 shlab-handout]$ make rtest10
./sdriver.pl -t trace10.txt -s ./tshref -a "-p"
#
# trace10.txt - Process fg builtin command.
#
tsh> ./myspin 4 &
[1] (12207) ./myspin 4 &
tsh> fg %1
Job [1] (12207) stopped by signal 20
tsh> jobs
[1] (12207) Stopped ./myspin 4 &
tsh> fg %1
tsh> jobs
```

K. Trace11

```
[hyjang@programming2 shlab-handout]$ make test11
./sdriver.pl -t trace11.txt -s ./tsh -a "-p"
#
# trace11.txt - Forward SIGINT to every process in foreground process group
#
tsh> ./mysplit 4
Job [1] (12361) terminated by signal 2
tsh> /bin/ps a
  PID TTY          STAT TIME COMMAND
  1621 ttty1    Ss   0:00 -bash
  1648 ttty1    S    0:00 su -
  1649 ttty1    S+   0:00 -bash
  7703 pts/2    Ss+  0:00 /bin/bash -l
  8517 pts/5    Ss+  0:00 /bin/bash -l
  8552 pts/6    Ss+  0:00 /bin/bash -l
  10280 pts/17   Ss   0:00 -bash
  10319 pts/17   S+   0:00 ssh hyjang@programming2.postech.ac.kr -p2022
  10326 pts/18   Ss   0:00 -bash
  12357 pts/18   S+   0:00 make test11
  12358 pts/18   S+   0:00 /usr/bin/perl ./sdriver.pl -t trace11.txt -s ./tsh -a -p
  12359 pts/18   S+   0:00 ./tsh -p
  12365 pts/18   R    0:00 /bin/ps a
  18083 pts/10   Ss+  0:00 -bash

[hyjang@programming2 shlab-handout]$ make rtest11
./sdriver.pl -t trace11.txt -s ./tshref -a "-p"
#
# trace11.txt - Forward SIGINT to every process in foreground process group
#
tsh> ./mysplit 4
Job [1] (12381) terminated by signal 2
tsh> /bin/ps a
  PID TTY          STAT TIME COMMAND
  1621 ttty1    Ss   0:00 -bash
  1648 ttty1    S    0:00 su -
  1649 ttty1    S+   0:00 -bash
  7703 pts/2    Ss+  0:00 /bin/bash -l
  8517 pts/5    Ss+  0:00 /bin/bash -l
  8552 pts/6    Ss+  0:00 /bin/bash -l
  10280 pts/17   Ss   0:00 -bash
  10319 pts/17   S+   0:00 ssh hyjang@programming2.postech.ac.kr -p2022
  10326 pts/18   Ss   0:00 -bash
  12376 pts/18   S+   0:00 make rtest11
  12377 pts/18   S+   0:00 /usr/bin/perl ./sdriver.pl -t trace11.txt -s ./tshref -a -p
  12379 pts/18   S+   0:00 ./tshref -p
  12385 pts/18   R    0:00 /bin/ps a
  18083 pts/10   Ss+  0:00 -bash
```

L. Trace12

```
[hyjang@programming2 shlab-handout]$ make test12
./sdriver.pl -t trace12.txt -s ./tsh -a "-p"
#
# trace12.txt - Forward SIGTSTP to every process in foreground process group
#
tsh> ./mysplit 4
Job [1] (12462) stopped by signal 20
tsh> jobs
[1] (12462) Stopped ./mysplit 4
tsh> /bin/ps a
  PID TTY          STAT TIME COMMAND
  1621 ttty1    Ss   0:00 -bash
  1648 ttty1    S    0:00 su -
  1649 ttty1    S+   0:00 -bash
  7703 pts/2    Ss+  0:00 /bin/bash -l
  8517 pts/5    Ss+  0:00 /bin/bash -l
  8552 pts/6    Ss+  0:00 /bin/bash -l
  10280 pts/17   Ss   0:00 -bash
  10319 pts/17   S+   0:00 ssh hyjang@programming2.postech.ac.kr -p2022
  10326 pts/18   Ss   0:00 -bash
  12458 pts/18   S+   0:00 make test12
  12459 pts/18   S+   0:00 /usr/bin/perl ./sdriver.pl -t trace12.txt -s ./tsh -a -p
  12460 pts/18   S+   0:00 ./tsh -p
  12462 pts/18   T    0:00 ./mysplit 4
  12463 pts/18   T    0:00 ./mysplit 4
  12467 pts/18   R    0:00 /bin/ps a
  18083 pts/10   Ss+  0:00 -bash

[hyjang@programming2 shlab-handout]$ make rtest12
./sdriver.pl -t trace12.txt -s ./tshref -a "-p"
#
# trace12.txt - Forward SIGTSTP to every process in foreground process group
#
tsh> ./mysplit 4
Job [1] (12497) stopped by signal 20
tsh> jobs
[1] (12497) Stopped ./mysplit 4
tsh> /bin/ps a
  PID TTY          STAT TIME COMMAND
  1621 ttty1    Ss   0:00 -bash
  1648 ttty1    S    0:00 su -
  1649 ttty1    S+   0:00 -bash
  7703 pts/2    Ss+  0:00 /bin/bash -l
  8517 pts/5    Ss+  0:00 /bin/bash -l
  8552 pts/6    Ss+  0:00 /bin/bash -l
  10280 pts/17   Ss   0:00 -bash
  10319 pts/17   S+   0:00 ssh hyjang@programming2.postech.ac.kr -p2022
  10326 pts/18   Ss   0:00 -bash
  12493 pts/18   S+   0:00 make rtest12
  12494 pts/18   S+   0:00 /usr/bin/perl ./sdriver.pl -t trace12.txt -s ./tshref -a -p
  12495 pts/18   S+   0:00 ./tshref -p
  12497 pts/18   T    0:00 ./mysplit 4
  12498 pts/18   T    0:00 ./mysplit 4
  12502 pts/18   R    0:00 /bin/ps a
  18083 pts/10   Ss+  0:00 -bash
```

M. Trace13

```
[hyjang@programming2 shlab-handout]$ make test13
./sdriver.pl -t trace13.txt -s ./tsh -a "-p"
#
# trace13.txt - Restart every stopped process in process group
#
tsh> ./mysplit 4
Job [1] (12572) stopped by signal 20
tsh> jobs
[1] (12572) Stopped ./mysplit 4
tsh> /bin/ps a
  PID TTY          STAT TIME COMMAND
 1621 tty1        Ss   0:00 -bash
 1648 tty1        S    0:00 su -
 1649 tty1        S+   0:00 -bash
 7703 pts/2       Ss+   0:00 /bin/bash -l
 8517 pts/5       Ss+   0:00 /bin/bash -l
 8552 pts/6       Ss+   0:00 /bin/bash -l
10280 pts/17     Ss   0:00 -bash
10319 pts/17     S+   0:00 ssh hyjang@programming2.postech.ac.kr -p2022
10326 pts/18     Ss   0:00 -bash
12568 pts/18     S+   0:00 make test13
12569 pts/18     S+   0:00 /usr/bin/perl ./sdriver.pl -t trace13.txt -s ./tsh -a -p
12570 pts/18     S+   0:00 ./tsh -p
12572 pts/18     T    0:00 ./mysplit 4
12573 pts/18     T    0:00 ./mysplit 4
12577 pts/18     R    0:00 /bin/ps a
18083 pts/10     Ss+   0:00 -bash
tsh> fg %1
tsh> /bin/ps a
  PID TTY          STAT TIME COMMAND
 1621 tty1        Ss   0:00 -bash
 1648 tty1        S    0:00 su -
 1649 tty1        S+   0:00 -bash
 7703 pts/2       Ss+   0:00 /bin/bash -l
 8517 pts/5       Ss+   0:00 /bin/bash -l
 8552 pts/6       Ss+   0:00 /bin/bash -l
10280 pts/17     Ss   0:00 -bash
10319 pts/17     S+   0:00 ssh hyjang@programming2.postech.ac.kr -p2022
10326 pts/18     Ss   0:00 -bash
12568 pts/18     S+   0:00 make test13
12569 pts/18     S+   0:00 /usr/bin/perl ./sdriver.pl -t trace13.txt -s ./tsh -a -p
12570 pts/18     S+   0:00 ./tsh -p
12572 pts/18     T    0:00 ./mysplit 4
12573 pts/18     T    0:00 ./mysplit 4
12577 pts/18     R    0:00 /bin/ps a
18083 pts/10     Ss+   0:00 -bash
tsh> fg %1
tsh> /bin/ps a
  PID TTY          STAT TIME COMMAND
 1621 tty1        Ss   0:00 -bash
 1648 tty1        S    0:00 su -
 1649 tty1        S+   0:00 -bash
 7703 pts/2       Ss+   0:00 /bin/bash -l
 8517 pts/5       Ss+   0:00 /bin/bash -l
 8552 pts/6       Ss+   0:00 /bin/bash -l
10280 pts/17     Ss   0:00 -bash
10319 pts/17     S+   0:00 ssh hyjang@programming2.postech.ac.kr -p2022
10326 pts/18     Ss   0:00 -bash
12568 pts/18     S+   0:00 make rtest13
12569 pts/18     S+   0:00 /usr/bin/perl ./sdriver.pl -t trace13.txt -s ./tshref -a -p
12589 pts/18     S+   0:00 ./tshref -p
12591 pts/18     T    0:00 ./mysplit 4
12592 pts/18     T    0:00 ./mysplit 4
12596 pts/18     R    0:00 /bin/ps a
18083 pts/10     Ss+   0:00 -bash
tsh> fg %1
tsh> /bin/ps a
  PID TTY          STAT TIME COMMAND
 1621 tty1        Ss   0:00 -bash
 1648 tty1        S    0:00 su -
 1649 tty1        S+   0:00 -bash
 7703 pts/2       Ss+   0:00 /bin/bash -l
 8517 pts/5       Ss+   0:00 /bin/bash -l
 8552 pts/6       Ss+   0:00 /bin/bash -l
10280 pts/17     Ss   0:00 -bash
10319 pts/17     S+   0:00 ssh hyjang@programming2.postech.ac.kr -p2022
10326 pts/18     Ss   0:00 -bash
12568 pts/18     S+   0:00 make rtest13
12569 pts/18     S+   0:00 /usr/bin/perl ./sdriver.pl -t trace13.txt -s ./tshref -a -p
12589 pts/18     S+   0:00 ./tshref -p
12591 pts/18     T    0:00 ./mysplit 4
12592 pts/18     T    0:00 ./mysplit 4
12596 pts/18     R    0:00 /bin/ps a
18083 pts/10     Ss+   0:00 -bash
```

N. Trace14

```
[hyjang@programming2 shlab-handout]$ make test14
./sdriver.pl -t trace14.txt -s ./tsh -a "-p"
#
# trace14.txt - Simple error handling
#
tsh> ./bogus
./bogus: Command not found
tsh> ./myspin 4 &
[1] (14288) ./myspin 4 &
tsh> fg
fg command requires PID or %jobid argument
tsh> bg
bg command requires PID or %jobid argument
tsh> fg a
fg: argument must be a PID or %jobid
tsh> bg a
bg: argument must be a PID or %jobid
tsh> fg 99999999
(99999999): No such process
tsh> bg 99999999
(99999999): No such process
tsh> fg %2
%2: No such job
tsh> fg %1
Job [1] (14288) stopped by signal 20
tsh> bg %2
%2: No such job
tsh> bg %1
[1] (14288) ./myspin 4 &
tsh> jobs
[1] (14288) Running ./myspin 4 &

[hyjang@programming2 shlab-handout]$ make rtest14
./sdriver.pl -t trace14.txt -s ./tshref -a "-p"
#
# trace14.txt - Simple error handling
#
tsh> ./bogus
./bogus: Command not found
tsh> ./myspin 4 &
[1] (14313) ./myspin 4 &
tsh> fg
fg command requires PID or %jobid argument
tsh> bg
bg command requires PID or %jobid argument
tsh> fg a
fg: argument must be a PID or %jobid
tsh> bg a
bg: argument must be a PID or %jobid
tsh> fg 99999999
(99999999): No such process
tsh> bg 99999999
(99999999): No such process
tsh> fg %2
%2: No such job
tsh> fg %1
Job [1] (14313) stopped by signal 20
tsh> bg %2
%2: No such job
tsh> bg %1
[1] (14313) ./myspin 4 &
tsh> jobs
[1] (14313) Running ./myspin 4 &
```

O. Trace15

```
[hyjang@programming2 shlab-handout]$ make test15
./sdriver.pl -t trace15.txt -s ./tsh -a "-p"
#
# trace15.txt - Putting it all together
#
tsh> ./bogus
./bogus: Command not found
tsh> ./myspin 10
Job [1] (14502) terminated by signal 2
tsh> ./myspin 3 &
[1] (14513) ./myspin 3 &
tsh> ./myspin 4 &
[2] (14515) ./myspin 4 &
tsh> jobs
[1] (14513) Running ./myspin 3 &
[2] (14515) Running ./myspin 4 &
tsh> fg %1
Job [1] (14513) stopped by signal 20
tsh> jobs
[1] (14513) Stopped ./myspin 3 &
[2] (14515) Running ./myspin 4 &
tsh> bg %3
%3: No such job
tsh> bg %1
[1] (14513) ./myspin 3 &
tsh> jobs
[1] (14513) Running ./myspin 3 &
[2] (14515) Running ./myspin 4 &
tsh> fg %1
tsh> quit

[hyjang@programming2 shlab-handout]$ make rtest15
./sdriver.pl -t trace15.txt -s ./tshref -a "-p"
#
# trace15.txt - Putting it all together
#
tsh> ./bogus
./bogus: Command not found
tsh> ./myspin 10
Job [1] (14542) terminated by signal 2
tsh> ./myspin 3 &
[1] (14546) ./myspin 3 &
tsh> ./myspin 4 &
[2] (14548) ./myspin 4 &
tsh> jobs
[1] (14546) Running ./myspin 3 &
[2] (14548) Running ./myspin 4 &
tsh> fg %1
Job [1] (14546) stopped by signal 20
tsh> jobs
[1] (14546) Stopped ./myspin 3 &
[2] (14548) Running ./myspin 4 &
tsh> bg %3
%3: No such job
tsh> bg %1
[1] (14546) ./myspin 3 &
tsh> jobs
[1] (14546) Running ./myspin 3 &
[2] (14548) Running ./myspin 4 &
tsh> fg %1
tsh> quit
```

P. Trace16

```
[hyjang@programming2 shlab-handout]$ make test16
./sdriver.pl -t trace16.txt -s ./tsh -a "-p"
#
# trace16.txt - Tests whether the shell can handle SIGTSTP and SIGINT
#               signals that come from other processes instead of the terminal.
#
tsh> ./mystop 2
Job [1] (14629) stopped by signal 20
tsh> jobs
[1] (14629) Stopped ./mystop 2
tsh> ./myint 2
Job [2] (14635) terminated by signal 2
[hyjang@programming2 shlab-handout]$ make rtest16
./sdriver.pl -t trace16.txt -s ./tshref -a "-p"
#
# trace16.txt - Tests whether the shell can handle SIGTSTP and SIGINT
#               signals that come from other processes instead of the terminal.
#
tsh> ./mystop 2
Job [1] (14645) stopped by signal 20
tsh> jobs
[1] (14645) Stopped ./mystop 2
tsh> ./myint 2
Job [2] (14651) terminated by signal 2
```

10. Reference

[CSED 211] lecture 13 Exceptions and Processes

[CSED 211] lecture 14 Signals

[CSED 211] Shell lab 문서

writeup문서

Computer Systems: A Programmer's Perspective(교과서)